


```
import numpy as np
import matplotlib.pyplot as plt

# Inputs
x1, x2 = 0.4, 0.2
y = 1

# Parameters
w1, w2, b = 0.4, -0.5, 0.2
lr = 1

# Sigmoid
def sigmoid(z):
    return 1/(1+np.exp(-z))

# Forward pass
z = w1*x1 + w2*x2 + b
y_pred = sigmoid(z)

# Loss derivative
dL_dy_pred = y_pred - y

# Sigmoid derivative
Dy_pred_dz = y_pred*(1-y_pred)

# Chain rule
dL_dz = dL_dy_pred * Dy_pred_dz

# Gradients
dL_dw1 = dL_dz * x1
dL_dw2 = dL_dz * x2
dL_db = dL_dz

# Update
w1 -= lr*dL_dw1
w2 -= lr*dL_dw2
b -= lr*dL_db

print(w1, w2, b)
```

```
0.442808817074409 -0.4785955914627955 0.3070220426860225
```

```
import numpy as np

# AND data
X = np.array([
    [0,0],
    [0,1],
    [1,0].
```

```

    [1,1]
])

y = np.array([0,0,0,1])

w = np.array([0.1, 0.2])
b = 0.3
lr = 0.1

def sigmoid(z):
    return 1/(1+np.exp(-z))

for epoch in range(1000):
    for i in range(len(X)):
        z = np.dot(X[i], w) + b
        y_pred = sigmoid(z)

        dz = y_pred - y[i]
        dw = dz * X[i]
        db = dz
        w -= lr * dw
        b -= lr * db

print("Weights:", w)
print("Bias:", b)
print("my name is ajay adhikari")

```

```

Weights: [5.60154076 5.59544869]
Bias: -8.565987677634602

```

```

import numpy as np
import matplotlib.pyplot as plt

# Inputs
x1, x2 = 0.6, 0.1
y = 1

# Parameters
w1, w2, b = 0.2, -0.3, 0.4
lr = 0.1

# Sigmoid
def sigmoid(z):
    return 1/(1+np.exp(-z))

# To store loss values
loss_list = []

```

```

# Training for multiple epochs
epochs = 1000

for epoch in range(epochs):

    # Forward pass
    z = w1*x1 + w2*x2 + b
    y_pred = sigmoid(z)

    # Loss (Binary Cross Entropy)
    loss = -(y*np.log(y_pred) + (1-y)*np.log(1-y_pred))
    loss_list.append(loss)

    # Backpropagation
    dL_dy_pred = y_pred - y
    Dy_pred_dz = y_pred * (1 - y_pred)
    dL_dz = dL_dy_pred * Dy_pred_dz

    dL_dw1 = dL_dz * x1
    dL_dw2 = dL_dz * x2
    dL_db = dL_dz

    # Update
    w1 -= lr * dL_dw1
    w2 -= lr * dL_dw2
    b -= lr * dL_db

# Final parameters
print("Final w1:", w1)
print("Final w2:", w2)
print("Final b :", b)

# Plotting loss graph
plt.plot(loss_list)
plt.xlabel("Epochs")
plt.ylabel("Loss (Error)")
plt.title("Error Reduction Graph (Loss vs Epochs)")
plt.grid(True)
plt.show()

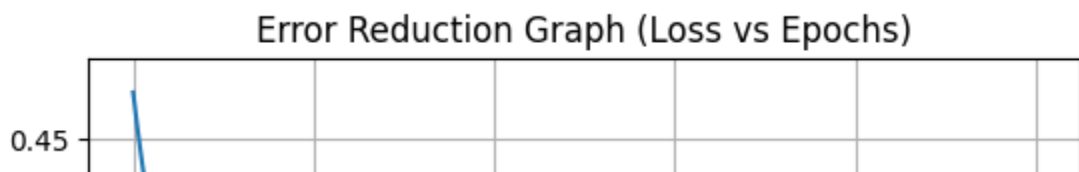
print("AJAY ADHIKARI, 428")
print("Date:05/02/2026")

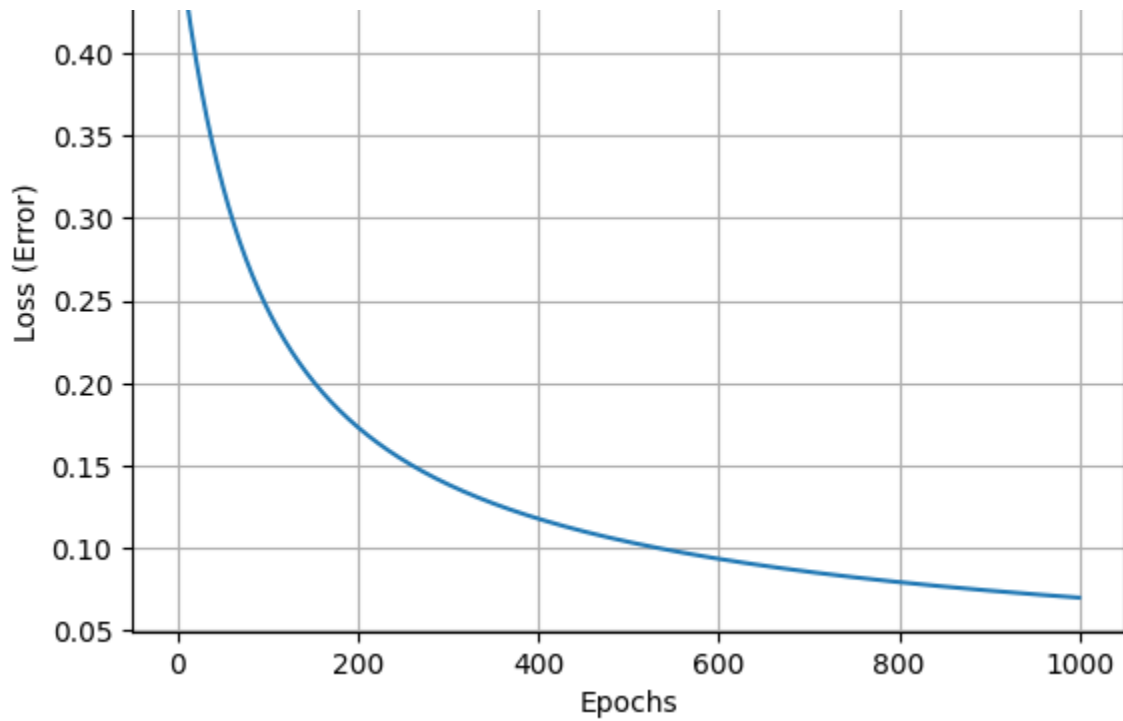
```

```

Final w1: 1.134656047343927
Final w2: -0.14422399210934486
Final b : 1.9577600789065457

```





AJAY ADHIKARI, 428

Date:05/02/2026

```
import numpy as np
import matplotlib.pyplot as plt

# Inputs
x1, x2 = 0.6, 0.1
y = 1

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Learning rates to compare
learning_rates = [0.001, 0.05, 0.1, 0.5, 1.0]

epochs = 1000

# Plot setup
plt.figure()

for lr in learning_rates:

    # Initialize parameters (same start for fair comparison)
    w1, w2, b = 0.2, -0.3, 0.4
    loss_list = []

    for epoch in range(epochs):

        # Forward pass
```

```

z = w1*x1 + w2*x2 + b
y_pred = sigmoid(z)

# Binary Cross Entropy Loss
loss = -(y*np.log(y_pred) + (1-y)*np.log(1-y_pred))
loss_list.append(loss)

# Backpropagation
dL_dy_pred = y_pred - y
dy_pred_dz = y_pred * (1 - y_pred)
dL_dz = dL_dy_pred * dy_pred_dz

dL_dw1 = dL_dz * x1
dL_dw2 = dL_dz * x2
dL_db = dL_dz

# Update parameters
w1 -= lr * dL_dw1
w2 -= lr * dL_dw2
b -= lr * dL_db

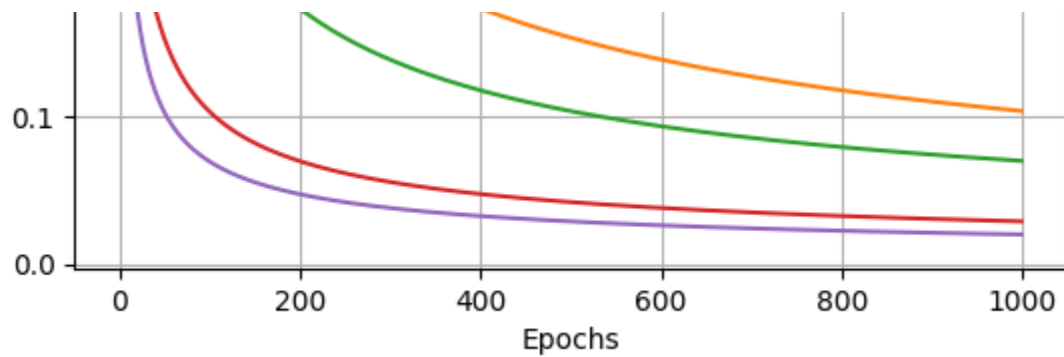
# Plot loss curve for this learning rate
plt.plot(loss_list, label=f"LR = {lr}")

# Graph details
plt.xlabel("Epochs")
plt.ylabel("Loss (Error)")
plt.title("Comparison of Learning Rates (Loss vs Epochs)")
plt.legend()
plt.grid(True)
plt.show()

print("AJAY ADHIKARI, 428")
print("Date: 05/02/2026")

```





AJAY ADHIKARI, 428

Date: 05/02/2026

```
import numpy as np
import matplotlib.pyplot as plt

# AND gate dataset
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 0, 0, 1])

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Learning rates to compare
learning_rates = [0.01, 0.05, 0.1, 0.5, 1.0]

epochs = 1000

plt.figure()

for lr in learning_rates:

    # Initialize weights and bias
    w = np.array([0.2, -0.3])
    b = 0.4

    loss_list = []

    for epoch in range(epochs):

        total_loss = 0

        for i in range(len(X)):
```

```

x1, x2 = X[i]
target = y[i]

# Forward pass
z = w[0]*x1 + w[1]*x2 + b
y_pred = sigmoid(z)

# Binary Cross Entropy Loss
loss = -(target*np.log(y_pred) + (1-target)*np.log(1-y_pred))
total_loss += loss

# Backpropagation
dL_dy_pred = y_pred - target
dy_pred_dz = y_pred * (1 - y_pred)
dL_dz = dL_dy_pred * dy_pred_dz

dL_dw1 = dL_dz * x1
dL_dw2 = dL_dz * x2
dL_db = dL_dz

# Update weights
w[0] -= lr * dL_dw1
w[1] -= lr * dL_dw2
b -= lr * dL_db

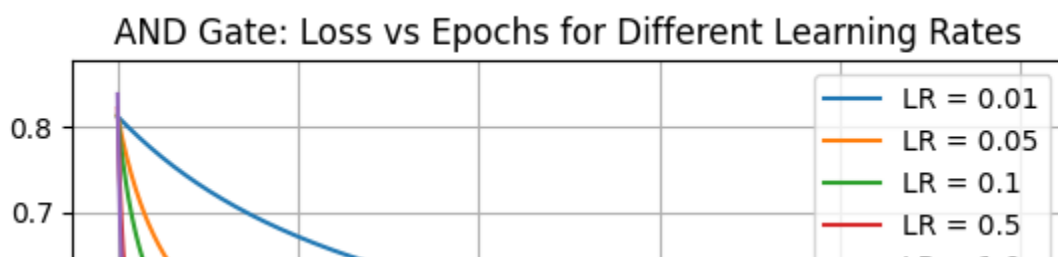
# Average loss per epoch
loss_list.append(total_loss / len(X))

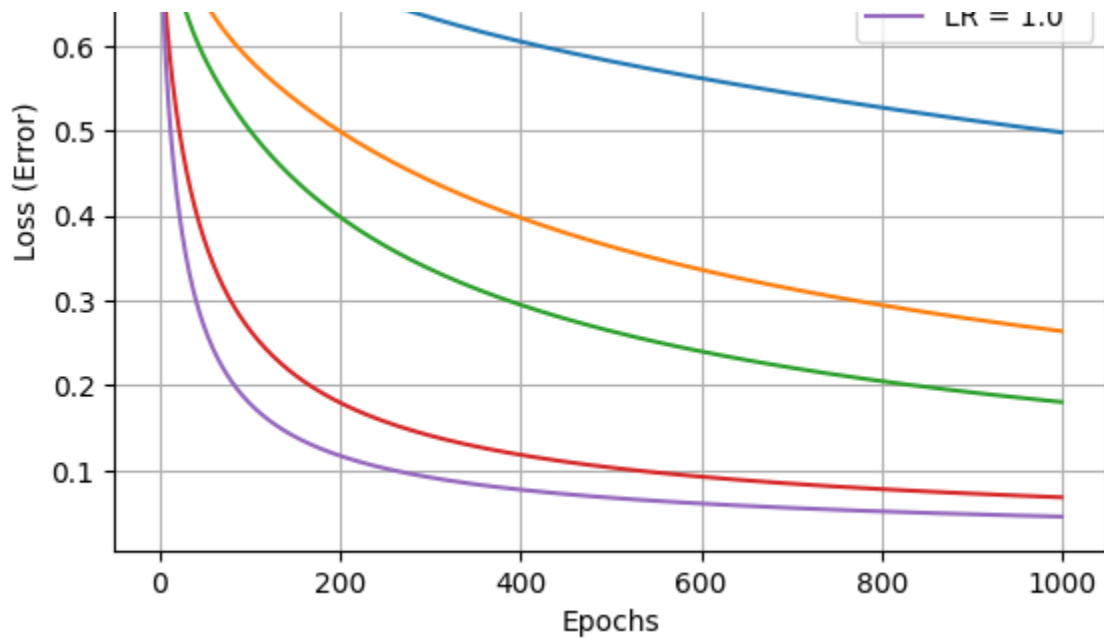
# Plot loss curve
plt.plot(loss_list, label=f"LR = {lr}")

# Graph details
plt.xlabel("Epochs")
plt.ylabel("Loss (Error)")
plt.title("AND Gate: Loss vs Epochs for Different Learning Rates")
plt.legend()
plt.grid(True)
plt.show()

print("AJAY ADHIKARI, 428")
print("Date: 05/02/2026")

```





AJAY ADHIKARI, 428
Date: 05/02/2026

```
import numpy as np
import matplotlib.pyplot as plt

# OR gate dataset
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 1])

# Sigmoid activation
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Learning rates to compare
learning_rates = [0.01, 0.05, 0.1, 0.5, 1.0]

epochs = 1000

plt.figure()

for lr in learning_rates:
    # Initialize weights and bias (same start)
    w = np.array([0.2, -0.3])
    b = 0.4
```

```
loss_list = []

for epoch in range(epochs):

    total_loss = 0

    for i in range(len(X)):

        x1, x2 = X[i]
        target = y[i]

        # Forward pass
        z = w[0]*x1 + w[1]*x2 + b
        y_pred = sigmoid(z)

        # Binary Cross Entropy Loss
        loss = -(target*np.log(y_pred) + (1-target)*np.log(1-y_pred))
        total_loss += loss

        # Backpropagation
        dL_dy_pred = y_pred - target
        dy_pred_dz = y_pred * (1 - y_pred)
        dL_dz = dL_dy_pred * dy_pred_dz

        dL_dw1 = dL_dz * x1
        dL_dw2 = dL_dz * x2
        dL_db = dL_dz

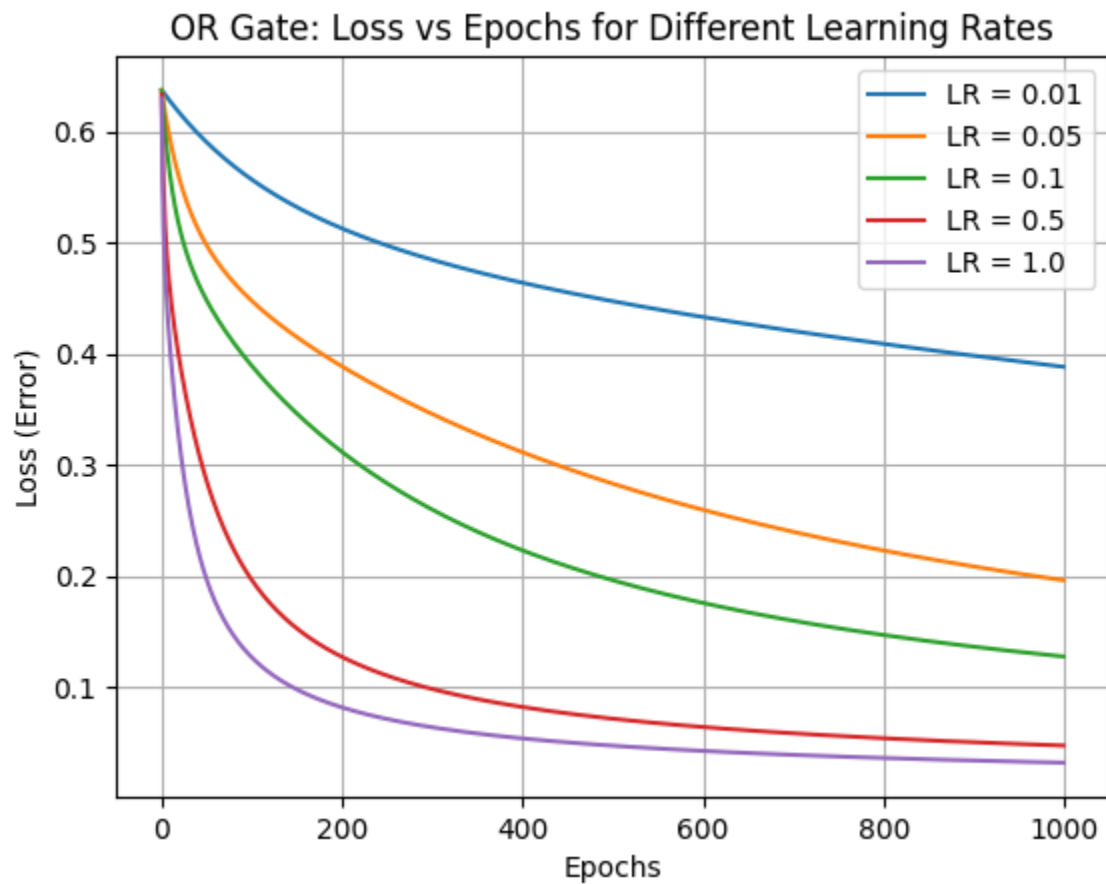
        # Update parameters
        w[0] -= lr * dL_dw1
        w[1] -= lr * dL_dw2
        b    -= lr * dL_db

    # Average loss
    loss_list.append(total_loss / len(X))

# Plot loss curve
plt.plot(loss_list, label=f"LR = {lr}")

# Graph details
plt.xlabel("Epochs")
plt.ylabel("Loss (Error)")
plt.title("OR Gate: Loss vs Epochs for Different Learning Rates")
plt.legend()
plt.grid(True)
plt.show()

print("AJAY ADHIKARI, 428")
print("Date: 05/02/2026")
```



AJAY ADHIKARI, 428
Date: 05/02/2026

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# OR gate data
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 1])

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Train single neuron (fixed LR)
w = np.array([0.2, -0.3])
b = 0.4
lr = 0.1
epochs = 1000
```

```

epochs = 1000

# Training
for epoch in range(epochs):
    for i in range(len(X)):
        x1, x2 = X[i]
        target = y[i]

        z = w[0]*x1 + w[1]*x2 + b
        y_pred = sigmoid(z)

        dL_dy = y_pred - target
        dy_dz = y_pred * (1 - y_pred)
        dL_dz = dL_dy * dy_dz

        w[0] -= lr * dL_dz * x1
        w[1] -= lr * dL_dz * x2
        b    -= lr * dL_dz

# Create meshgrid
x1_vals = np.linspace(0, 1, 50)
x2_vals = np.linspace(0, 1, 50)
X1, X2 = np.meshgrid(x1_vals, x2_vals)

Z = sigmoid(w[0]*X1 + w[1]*X2 + b)

# 3D Plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X1, X2, Z, alpha=0.7)
ax.scatter(X[:,0], X[:,1], y, color='red', s=60)

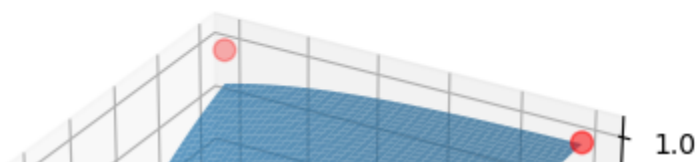
ax.set_xlabel("Input x1")
ax.set_ylabel("Input x2")
ax.set_zlabel("Output (ŷ)")
ax.set_title("3D Decision Surface for OR Gate")

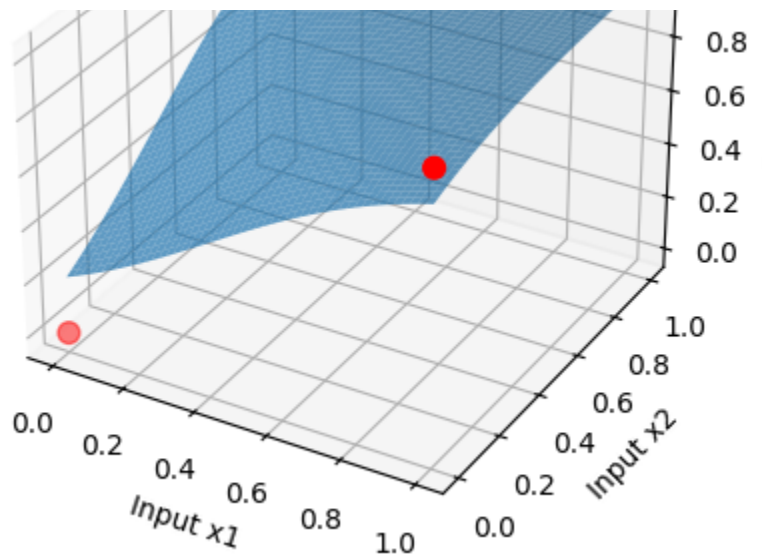
plt.show()

print("AJAY ADHIKARI, 428")
print("Date: 05/02/2026")

```

3D Decision Surface for OR Gate





AJAY ADHIKARI, 428

Date: 05/02/2026